

Data Structures (CS2103)

Prerequisites

Program, Programming, Compilation, Execution, Memory layout, Array, Pointers, Dynamic memory allocation etc.

Prepared By – Ashish K Layek
CST, IEST Shibpur

Computer Program

- ❑ A computer program (or just a program or software) is a sequence of instructions, written to perform a specified task with a computer.
 - ❑ A program is expected to behave in a predetermined manner. No matter how many times one program is run, the same result should be received for same set of data provided
-

Programming Language

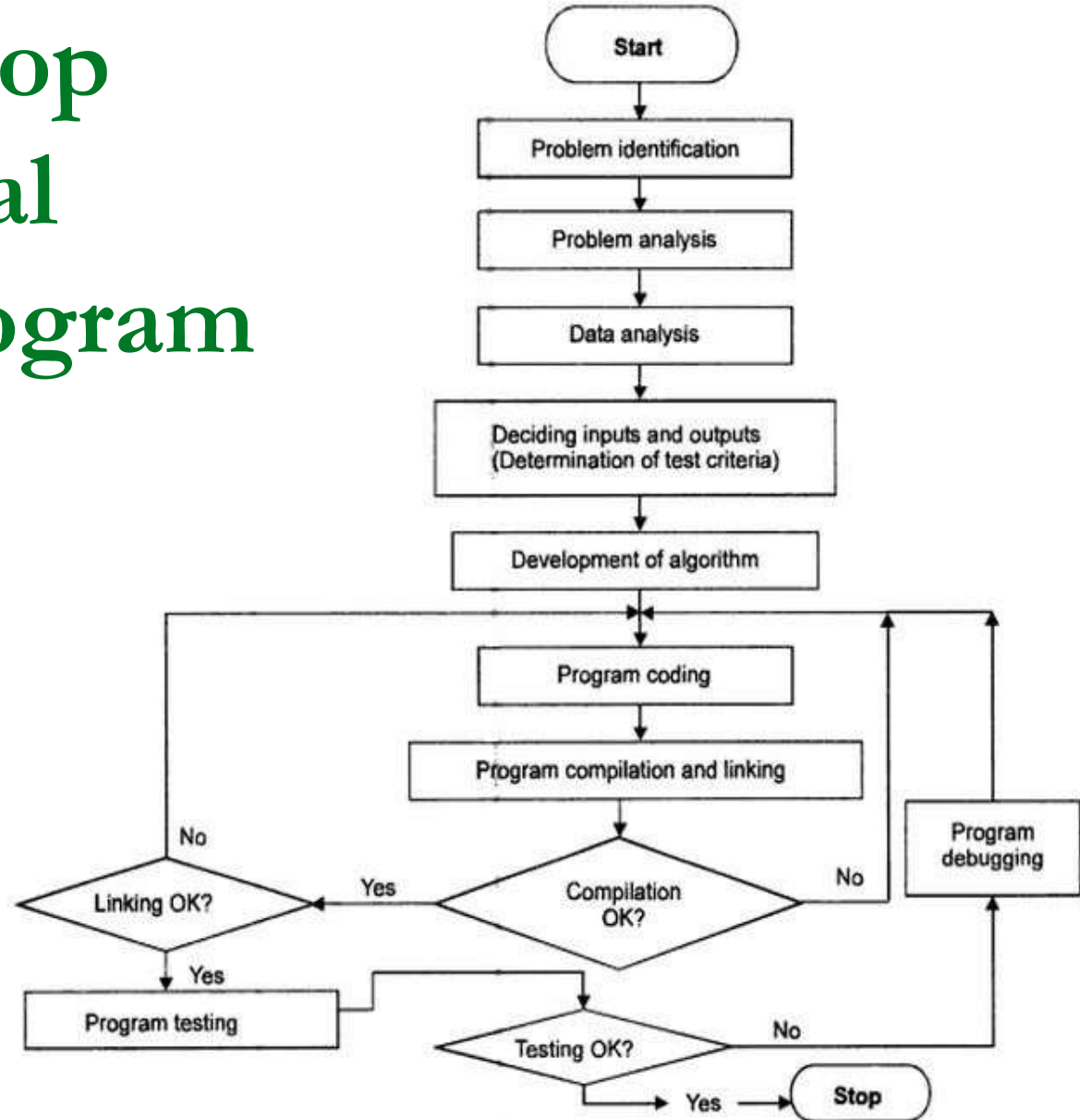
- ❑ Machine Language – Strict binary form / byte code
- ❑ High Level Language – C, Cobol, C++, Java, LISP etc

Note : High level languages are compiled or interpreted to **machine language** before execution

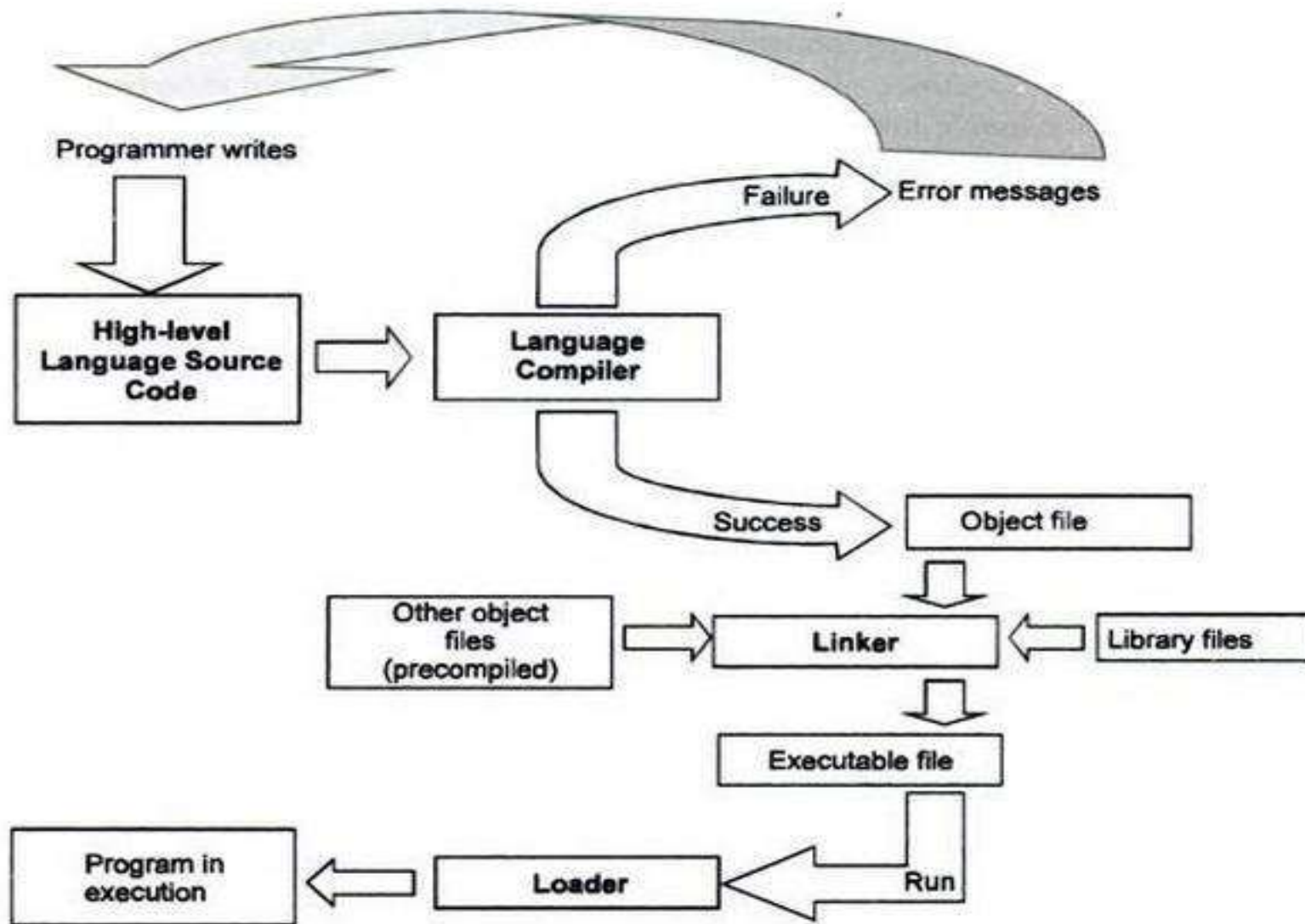
Computer Programming

- ❑ Computer programming is the iterative process of writing or editing source code that can be executed in a computer
 - ❑ It involves testing, analyzing, refining, and sometimes coordinating with other programmers on a jointly developed program
-

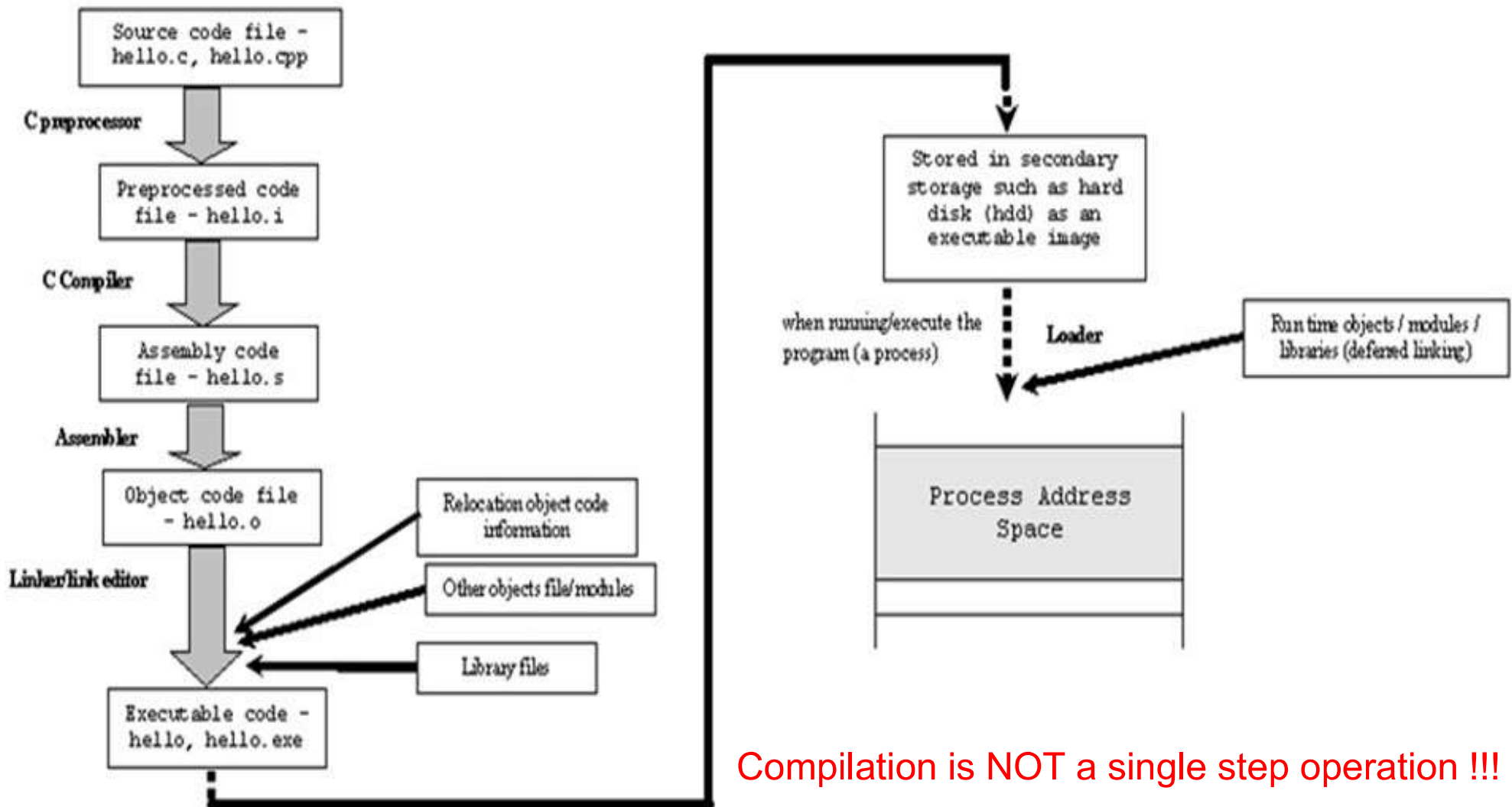
Steps to develop a Conventional Computer Program



Steps to execute a high level language program



Compilation of high level language program



Compilation

cont ...

`$> gcc hello.c -o hello` : the gcc compiler reads the source file `hello.c` and translates it into an executable `hello`. The compilation is performed in four sequential phases by the compilation system

- a collection of four programs – pre-processor, compiler, assembler, and linker.

`$> gcc -save-temps hello.c -o hello` : The “-save-temps” option will preserve and save all temporary files created during the C compilation. It will generate four files in the same directory namely

- `hello.i` (Generated by pre-processor)
- `hello.s` (Generated by compiler)
- `hello.o` (Generated by assembler)
- `hello` (Generated by linker)

Compilation

cont ...

Also can be observed as follows:

Step 1 – Pre-processing : `cpp hello.c > hello.i`

Step 2 – Compilation : `gcc -S hello.i`

Step 3 – Assembly : `as hello.s -o hello.o`

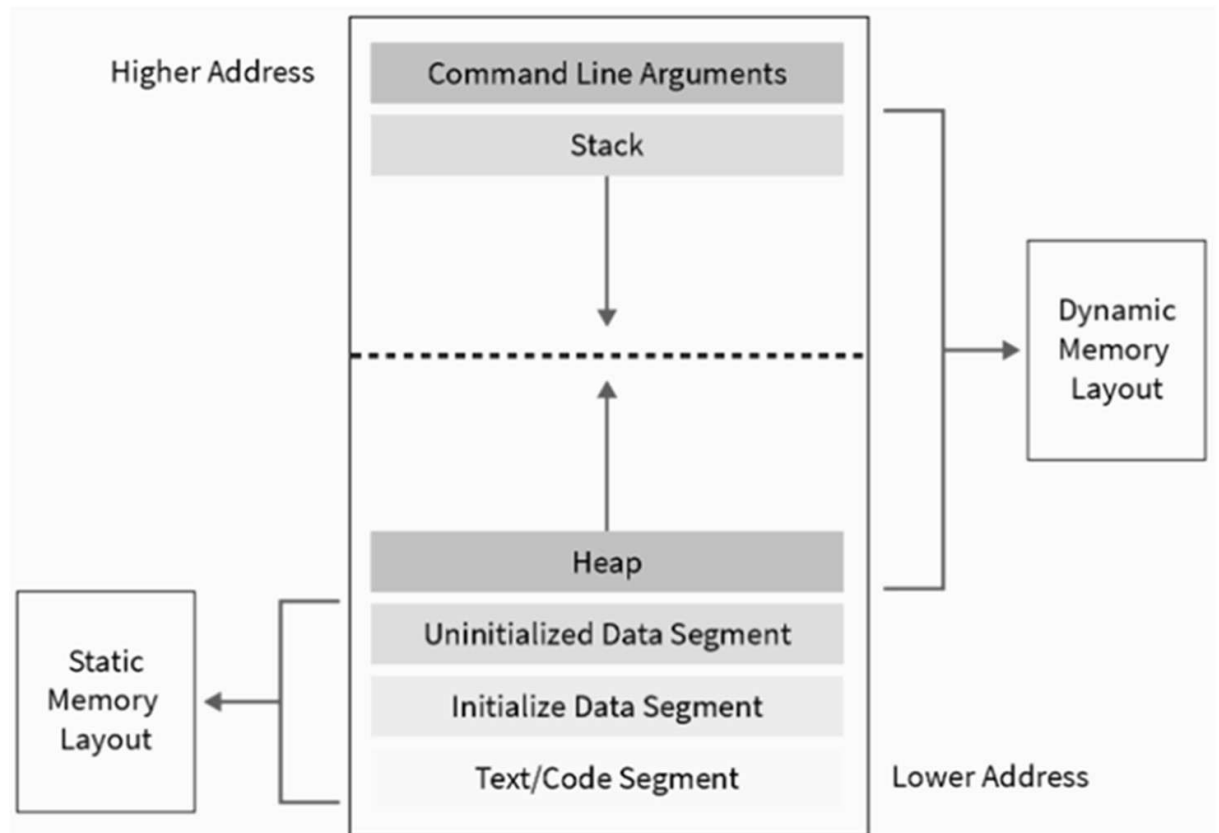
Step 4 – Linking : `ld -dynamic-linker /lib64/ld-linux-x86-64.so.2 /usr/lib64/crt1.o /usr/lib64/crti.o /usr/lib64/crtn.o helloworld.o /usr/lib/gcc/x86_64-redhat-linux/4.1.2/crtbegin.o -L /usr/lib/gcc/x86_64-redhat-linux/4.1.2/ -lgcc -lgcc_eh -lc -lgcc -lgcc_eh /usr/lib/gcc/x86_64-redhat-linux/4.1.2/crtend.o -o helloworld`

(This can be different from platform to platform)

C Programming Memory Layout

❑ In C programming, the memory layout consists of the following sections:

- Text segment
- Data segment
 - ✓ Uninitialized data
 - ✓ Initialized data
- Heap segment
- Stack segment



Arrays in C

All elements of same type – homogenous

Unlike Java, array size in declaration

```
int array[10];  
int b;
```

Compare: C: `int array[10];`

Java: `int[] array = new int[10];`

```
array[0] = 3;
```

First element (index 0)

```
array[9] = 4;
```

Last element (index size - 1)

```
array[10] = 5;
```

```
array[-1] = 6;
```

No bounds checking!

Allowed – usually causes no *obvious* error

array[10] may overwrite b

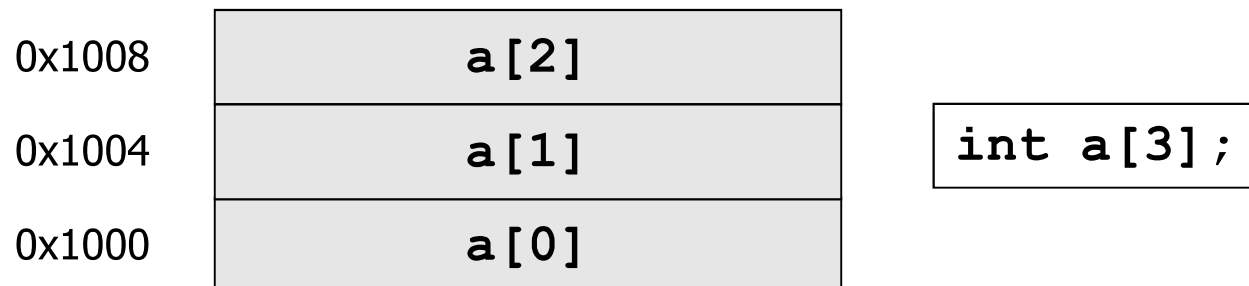
Array Representation

❑ Homogeneous → Each element same size – s bytes

- An array of m data values is a sequence of $m \times s$ bytes
- Indexing: 0th value at byte $s \times 0$, 1st value at byte $s \times 1$, ...

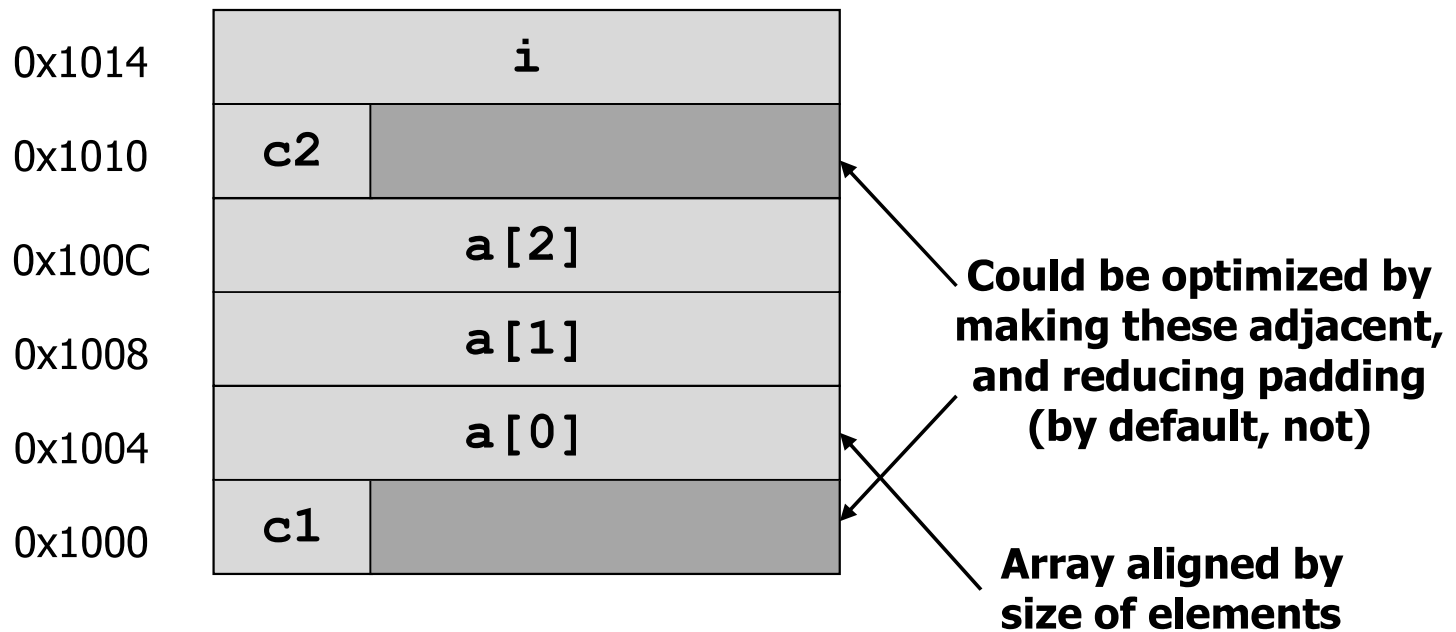
❑ m and s are not part of representation

- Unlike in some other languages
- s known by compiler – usually irrelevant to programmer
- m often known by compiler – if not, must be saved by programmer



Array Representation

```
char    c1;  
int     a[3];  
char    c2;  
int     i;
```



Array Sizes

```
int array[10];
```

What is

returns the size of
an object in bytes

 sizeof(array[3])? **4**

sizeof(array)? **40**

Multi-Dimensional Arrays

□ Row-Major Order (used in C/C++)

➤ $\text{Address}(A[i][j]) = \text{BA} + \text{element_size} \times (i \times \text{COLS} + j)$

Column-Major Order (used in Fortran, MATLAB)

➤ $\text{Address}(A[i][j]) = \text{BA} + \text{element_size} \times (j \times \text{ROWS} + i)$

- ✓ BA → Based Address
 - ✓ ROWS → Number of Rows
 - ✓ COLS → Number of Columns
-

Multi-Dimensional Arrays

```
int  matrix[2][3];  
  
matrix[1][0] = 17;
```

- **Recall: no bounds checking**
- **What happens when you write:**

```
matrix[0][3] = 42;
```

0x1014

`matrix[1][2]`

0x1010

`matrix[1][1]`

0x100C

`matrix[1][0]`

0x1008

`matrix[0][2]`

0x1004

`matrix[0][1]`

0x1000

`matrix[0][0]`

**“Row Major”
Organization**

Variable-Length Arrays

```
int
function(int n)
{
    int  array[n];
    ...
}
```

New C99 feature: Variable-length arrays defined within functions

Global arrays must still have fixed (constant) length

Memory Addresses

- ❑ Storage cells are typically viewed as being byte-sized
 - Usually the smallest addressable unit of memory
 - ✓ Few machines can directly address bits individually
 - Such addresses are sometimes called *byte-addresses*

 - ❑ Memory is often accessed as words
 - Usually a word is the largest unit of memory access by a single machine instruction
 - CLEAR's word size is 8 bytes (= `sizeof(long)`)
 - A *word-address* is simply the byte-address of the word's first byte
-

Pointers

- ❑ Special case of bounded-size natural numbers
 - Maximum memory limited by processor word-size
 - 2^{32} bytes = 4GB, 2^{64} bytes = 16 exabytes (EB)
- ❑ A pointer is just another kind of value
 - A basic type in C

```
int *ptr;
```

The variable “ptr” stores a pointer/address to an “int”.

Pointer Operations in C

❑ Creation

`& variable` Returns variable's memory address

❑ Dereference

`* pointer` Returns contents stored at address

❑ Indirect assignment

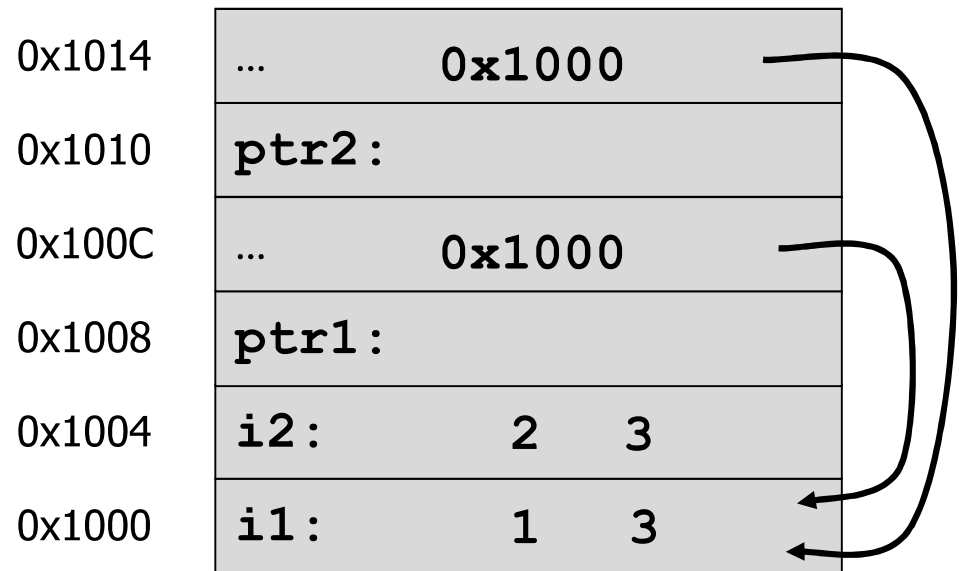
`* pointer = val` Stores value at address

❑ Assignment

`pointer = ptr` Stores pointer in another variable

Using Pointers

```
int i1;  
int i2;  
int *ptr1;  
int *ptr2;  
  
i1 = 1;  
i2 = 2;  
  
ptr1 = &i1;  
ptr2 = ptr1;  
  
*ptr1 = 3;  
i2 = *ptr2;
```



Using Pointers (cont.)

```
int  int1      = 1036;  /* some data to point to */
int  int2      = 8;

int *int_ptr1 = &int1;  /* get addresses of data */
int *int_ptr2 = &int2;

*int_ptr1 = int_ptr2;

*int_ptr1 = int2;
```

What happens?

Type check warning: `int_ptr2` is not an `int`

`int1` becomes 8

Using Pointers (cont.)

```
int  int1      = 1036;  /* some data to point to */
int  int2      = 8;

int *int_ptr1 = &int1;  /* get addresses of data */
int *int_ptr2 = &int2;

int_ptr1 = *int_ptr2;

int_ptr1 = int_ptr2;
```

What happens?

Type check warning: `*int_ptr2` is not an `int` *

Changes `int_ptr1` – doesn't change `int1`

Pointer Arithmetic

pointer + number

pointer - number

E.g., *pointer + 1* adds 1 something to a pointer

```
char *p;  
char a;  
char b;
```

```
p = &a;  
p += 1;
```

In each, p now points to b
(**Assuming compiler doesn't
reorder variables in memory**)

```
int *p;  
int a;  
int b;
```

```
p = &a;  
p += 1;
```

Adds $1 * \text{sizeof}(\text{char})$ to
the memory address

Adds $1 * \text{sizeof}(\text{int})$ to
the memory address

Pointer arithmetic should be used cautiously

A Special Pointer in C

❑ Special constant pointer `NULL`

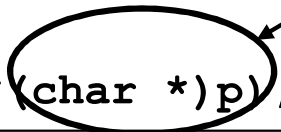
- Points to no data
- Dereferencing illegal – causes *segmentation fault*

To define, include `<stdlib.h>` or `<stdio.h>`

Generic Pointers

`void *`: a “pointer to anything”

```
void    *p;  
int     i;  
char    c;  
p = &i;  
p = &c;  
putchar(*(char *)p);
```



type cast: tells the compiler to “change” an object’s type (for type checking purposes – does not modify the object in any way)

Dangerous! Sometimes necessary...

❑ Lose all information about what type of thing is pointed to

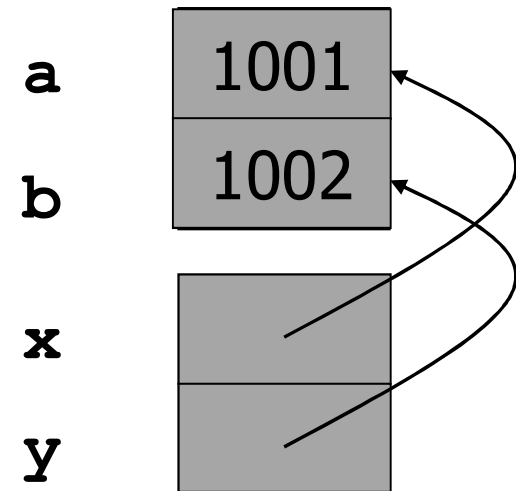
- Reduces effectiveness of compiler’s type-checking
- Can’t use pointer arithmetic

Pass-by-Address (Reference ?)

```
void
set_x_and_y(int *x, int *y)
{
    *x = 1001;
    *y = 1002;
}

void
f(void)
{
    int a = 1;
    int b = 2;

    set_x_and_y(&a, &b);
}
```



Arrays and Pointers

❖ Dirty “secret” :

- ❑ Array name $\approx$ a pointer to the initial (0th) array element

`a[i]     $\equiv$     *(a + i)`

- ❑ An array is passed to a function as a pointer
 - The array size is lost!
- ❑ Usually bad style to interchange arrays and pointers
 - Avoid pointer arithmetic!

Passing arrays:

Really `int *array` Must explicitly pass the size

```
int
foo(int array[],
    unsigned int size)
{
    ... array[size - 1] ...
}

int
main(void)
{
    int a[10], b[5];
    ... foo(a, 10)... foo(b, 5) ...
}
```

Arrays and Pointers

```
int
foo(int array[],
    unsigned int size)
{
    ...
    printf( "%d\n" , sizeof(array));
}

int
main(void)
{
    int a[10], b[5];
    ... foo(a, 10)... foo(b, 5) ...
    printf( "%d\n" , sizeof(a));
}
```

What does this print? **8**

... because `array` is really
a pointer

What does this print? **40**

Arrays and Pointers

```
int i;  
int array[10];  
  
for (i = 0; i < 10; i++) {  
    ...  
    array[i] = ...;  
    ...  
}
```

```
int *p;  
int array[10];  
  
for (p = array; p < &array[10]; p++) {  
    *p = ...;  
    ...  
}
```

These two blocks of code are functionally equivalent

Pointers and dynamic allocation

- ❑ C supports dynamic allocation (and deallocation) through two functions: `malloc()` and `free()`

Example code :

```
int *p = NULL;
p = (int *)malloc(sizeof(int));
*p = 20;
free(p);

int *q = (int*)malloc(10*sizeof(int));
int i = 0;
for(i = 0; i < 10; i++){
    q[i] = i; // or *(q+i) = i;
}
free(q);
```

Alias

Alias example code

```
int *p, *q;  
p = (int*)malloc(sizeof(int));  
*p = 20;  
q = p;    // q and p refer to the same location,  
          // they are now aliases  
*q = 30;  // even *p would have changed
```

Garbage or Memory Leak

Garbage example code

```
int *p = Null;
```

```
p = (int*)malloc(sizeof(int));
```

```
// Space for an int is
```

```
// create/reserved
```

```
*p = 20;
```

```
p = (int*)malloc(sizeof(int));
```

```
// AHA ! Space for one more int is
```

```
// created, earlier value and
```

```
// location lost - it's a memory
```

```
// leak and also called garbage
```

```
// location without accesses
```

Dangling reference

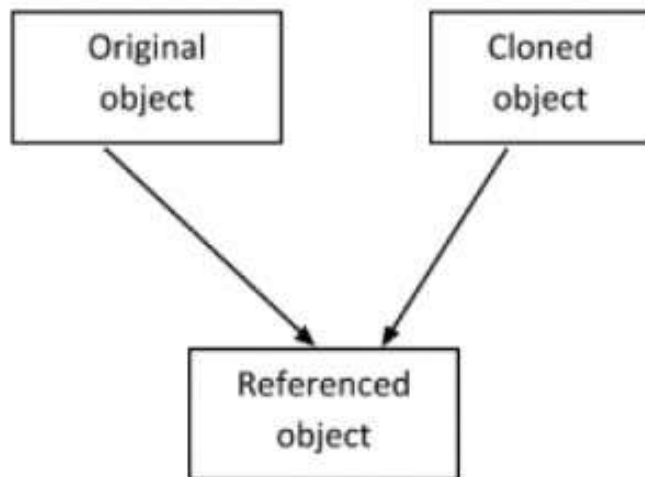
Dangling reference example code

```
int *p, *q;  
p = (int *)malloc(sizeof(int));  
*p = 22;  
q = p;    // q and p refer to the same location,  
          // they are now aliases  
free(p);  // space for int is released  
p = NULL;  
*q = 333; // AHA ! The pointer q points to a non-  
          // existing location. This is called  
          // dangling reference. This can causes  
          // memory corruption
```

This typical issue happens in case of shallow copy

Shallow copy vs Deep copy ?

Shallow copy is a bit-wise copy of an object. A new object is created that has an exact copy of the values in the original object. If any of the fields of the object are references to other objects, just the reference addresses are copied i.e., only the memory address is copied.



Deep copy copies all fields, and makes copies of dynamically allocated memory pointed to by the fields. A deep copy occurs when an object is copied along with the objects to which it refers.

